```
  int GetEmpid() const {return itsEmpid;}
  string GetName() const {return itsName;}
  string GetAddress() const {return itsAddress;}
  PaymentMethod* GetMethod() {return itsPaymentMethod;}
  PaymentClassification* GetClassification() {return itsClassification;}
  PaymentSchedule* GetSchedule() {return itsSchedule;}
  Affiliation* GetAffiliation() {return itsAffiliation;}

  void Payday(Paycheck&);
  bool IsPayDate(const Date& payDate) const;
  Date GetPayPeriodStartDate(const Date& payPeriodEndDate) const;

 private:
  int itsEmpid;
  string itsName;
  string itsAddress;
  PaymentClassification* itsClassification;
  PaymentSchedule* itsSchedule;
  PaymentMethod* itsPaymentMethod;
  Affiliation* itsAffiliation;
};

#endif
```

Listing 19-55

Employee.cpp

```
#include "Employee.h"
#include "NoAffiliation.h"
#include "PaymentClassification.h"
#include "PaymentSchedule.h"
#include "PaymentMethod.h"
#include "Paycheck.h"

Employee::~Employee()
{
  delete itsClassification;
  delete itsSchedule;
  delete itsPaymentMethod;
}

Employee::Employee(int empid, string name, string address)
: itsEmpid(empid)
, itsName(name)
, itsAddress(address)
, itsAffiliation(new NoAffiliation())
, itsClassification(0)
, itsSchedule(0)
, itsPaymentMethod(0)
{
}
```

```
void Employee::SetName(string name)
{
  itsName = name;
}

void Employee::SetAddress(string address)
{
  itsAddress = address;
}

void Employee::SetClassification(PaymentClassification* pc)
{
  delete itsClassification;
  itsClassification = pc;
}

void Employee::SetSchedule(PaymentSchedule* ps)
{
  delete itsSchedule;
  itsSchedule = ps;
}

void Employee::SetMethod(PaymentMethod* pm)
{
  delete itsPaymentMethod;
  itsPaymentMethod = pm;
}

void Employee::SetAffiliation(Affiliation* af)
{
  delete itsAffiliation;
  itsAffiliation = af;
}

bool Employee::IsPayDate(const Date& payDate) const
{
  return itsSchedule->IsPayDate(payDate);
}

Date Employee::GetPayPeriodStartDate(const Date& payPeriodEndDate) const
{
  return itsSchedule->GetPayPeriodStartDate(payPeriodEndDate);
}

void Employee::Payday(Paycheck& pc)
{
  Date payDate = pc.GetPayPeriodEndDate();
  double grossPay = itsClassification->CalculatePay(pc);
  double deductions = itsAffiliation->CalculateDeductions(pc);
  double netPay = grossPay - deductions;
  pc.SetGrossPay(grossPay);
  pc.SetDeductions(deductions);
  pc.SetNetPay(netPay);
  itsPaymentMethod->Pay(pc);
}
```

Main Program

The main payroll program can now be expressed as a loop that parses transactions from an input source and then executes them. Figures 19-34 and 19-35 describe the statics and dynamics of the main program. The concept is simple: the `PayrollApplication` sits in a loop, alternately requesting transactions from the `TransactionSource` and then telling those `Transaction` objects to `Execute`. Note that this is different from the diagram in Figure 19-1, and it represents a shift in our thinking to a more abstract mechanism.

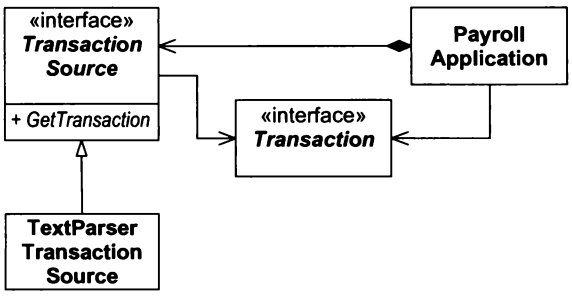


Figure 19-34 Static Model for the Main Program

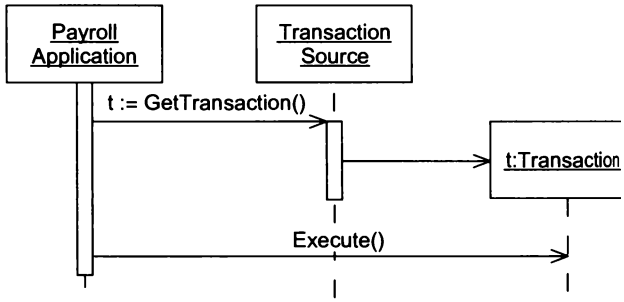


Figure 19-35 Dynamic Model for the Main Program

`TransactionSource` is an abstract class that we can implement in several ways. The static diagram shows the derivative named `TextParserTransactionSource`, which reads an incoming text stream and parses out the transactions as described in the use cases. This object then creates the appropriate `Transaction` objects and sends them along to the `PayrollApplication`.

The separation of interface from implementation in the `TransactionSource` allows the source of the transactions to be abstract; for example, we could easily interface the `PayrollApplication` to a `GUITransactionSource` or a `RemoteTransactionSource`.

The Database

Now that this iteration has been analyzed, designed, and (mostly) implemented, we can consider the role of the database. The class `PayrollDatabase` clearly encapsulates something involving persistence. The objects contained within the `PayrollDatabase` must live longer than any particular run of the application. How should this be implemented? Clearly the transient mechanism used by the test cases is not sufficient for the real system. We have several options.

We could implement `PayrollDatabase` using an object-oriented database management system (OODBMS). This would allow the actual objects to reside within the permanent storage of the database. As designers, we would have little more work to do, since the OODBMS would not add much new to our design. One of the great benefits of OODBMS products is that they have little or no impact on the object model of the applications. As far as the design is concerned, the database barely exists.[6]

Another option would be to use a simple, flat text file to record the data. Upon initialization, the `PayrollDatabase` object could read that file and build the necessary objects in memory. At the end of the program, the `PayrollDatabase` object could write a new version of the text file. Certainly this option would not suffice for a company with hundreds of thousands of employees, or for one that wanted real-time concurrent access to

6. This is optimistic. In a simple application like Payroll, the use of an OODBMS would have very little impact upon the design of the program. As applications become more and more complicated, the amount of impact that the OODBMS has upon the application increases. Still, the impact is far less than the impact an RDBMS would have.

its payroll database. However, it might suffice for a smaller company, and it could certainly be used as a mechanism for testing the rest of the application classes without investing in a big database system.

Still another option would be to incorporate a relational database management system (RDBMS) into the `PayrollDatabase` object. The implementation of the `PayrollDatabase` object would then make the appropriate queries to the RDMBS to temporarily create the necessary objects in memory.

The point is that, as far as the application is concerned, databases are simply mechanisms for managing storage. They should usually not be considered as a major factor of the design and implementation. As we have shown here, they can be left for last and handled as a detail.[7] By doing so, we leave open a number of interesting options for implementing the needed persistence and for creating mechanisms to test the rest of the application. We also do not tie ourselves to any particular database technology or product. We have the freedom to choose the database we need, based upon the rest of the design, and we maintain the freedom to change or replace that database product in the future as needed.

Summary of Payroll Design

In roughly 50 diagrams, and 3300 lines of code, we have shown the design and implementation of one iteration of the payroll application. The design employs a large amount of abstraction and polymorphism. The result is that large portions of the design are closed against changes of payroll policy. For example, the application could be changed to deal with employees who were paid quarterly based upon a normal salary and a bonus schedule. This change would require *addition* to the design, but little of the existing design and code would change.

During this process, we rarely considered whether we were performing analysis, design, or implementation. Instead, we concentrated upon issues of clarity and closure. We tried to find the underlying abstractions wherever possible. The result is that we have a good starting design for a payroll application, and we have a core of classes that are germane to the problem domain as a whole.

History

The diagrams in this chapter are derived from the Booch diagrams in the corresponding chapter of my 1995 book *Designing Object-Oriented C++ Applications using the Booch Method.* Those diagrams were created in 1994. As I created them, I also wrote some of the code that implemented them, to make sure that the diagrams made sense. However, I did not write anywhere near the amount of code presented here. Therefore, the diagrams did not benefit from significant feedback from the code and tests. This lack of feedback shows.

I wrote the current chapter in the order presented here. In every case, test cases were written before production code. In many cases, those tests were created incrementally, evolving as the production code also evolved. The production code was written to comply with the diagrams so long as that made sense. There were several cases where it did not make sense, and so I changed the design of the code.

One of the first places that this happened was back on page 219 when I decided against multiple `Affiliation` instances in the `Employee` object. Another was on page 229 when I found that I had not considered recording the employee's membership in the union in the `ChangeMemberTransaction`.

This is normal. When you design without feedback, you will necessarily make errors. It was the feedback imposed by the tests cases and running code that found these errors for us.

7. Sometimes the nature of the database is one of the requirements of the application. RDBMSs provide powerful query and reporting systems that may be listed as application requirements. However, even when such requirements are explicit, the designers should still decouple the application design from the database design. The application design should not have to depend on any particular kind of database.

Resources

You can find the final version of this code on the Prentice Hall Web site, or on `www.objectmentor.com/PPP`.

Bibliography

1. Jacobson, Ivar. *Object-Oriented Software Engineering, A Use-Case-Driven Approach*. Wokingham, UK: Addison–Wesley, 1992.

SECTION 4

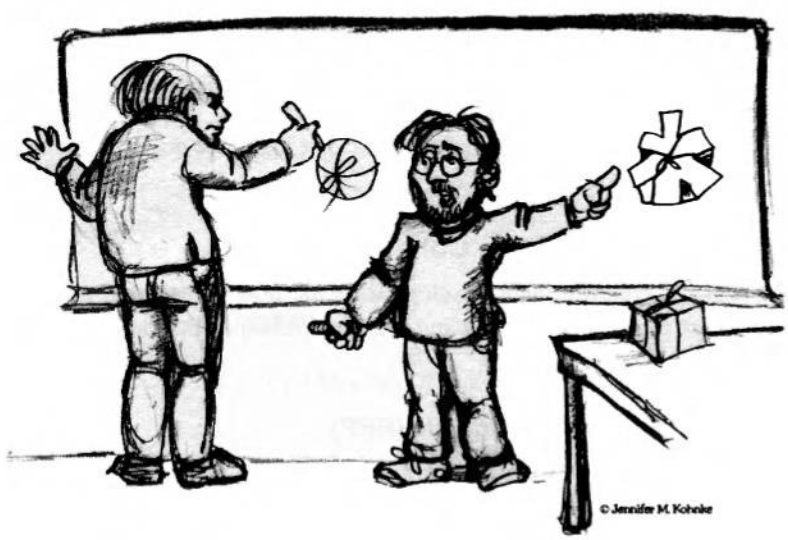
Packaging the Payroll System



In this section, we will explore the principles of design that help us split a large software system into packages. The first chapter in this section discusses those principles, the second describes a pattern that we'll use to help improve the packaging structure, and the third shows how the principles and pattern can be applied to the payroll system.

20

Principles of Package Design



Nice Package.

—Anthony

As software applications grow in size and complexity, they require some kind of high-level organization. Classes, while a very convenient unit for organizing small applications, are too finely grained to be used as the sole organizational unit for large applications. Something "larger" than a class is needed to help organize large applications. That something is called a *package*.

This chapter outlines six principles. The first three are principles of *package cohesion*. They help us allocate classes to packages. The last three principles govern *package coupling*. They help us determine how packages should be interrelated. The last two principles also describe a set of *Dependency Management (DM) metrics* that allows developers to measure and characterize the dependency structure of their designs.

Designing with Packages?

In UML, packages can be used as containers for groups of classes. By grouping classes into packages, we can reason about the design at a higher level of abstraction. We can also use the packages to manage the development and distribution of the software. The goal is to partition the classes in an application according to some criteria, and then allocate the classes in those partitions to packages.

But classes often have dependencies on other classes, and these dependencies will very often cross package boundaries. Thus, the packages will have dependency relationships with each other. The relationships between packages express the high-level organization of the application, and they need to be managed.

This begs a large number of questions.

1. What are the principles for allocating classes to packages?
2. What design principles govern the relationships between packages?
3. Should packages be designed before classes (top down)? Or should classes be designed before packages (bottom up)?
4. How are packages physically represented? In C++? In Java? In the development environment?
5. Once created, to what purpose will we put these packages?

This chapter presents six design principles that govern the creation, interrelationship, and use of packages. The first three govern the partitioning of classes into packages. The last three govern the interrelationships between packages.

Granularity: The Principles of Package Cohesion

The three principles of package cohesion help developers decide how to partition classes into packages. They depend on the fact that at least some of the classes and their interrelationships have been discovered. Thus, these principles take a "bottom-up" view of partitioning.

The Reuse–Release Equivalence Principle (REP)

The granule of reuse is the granule of release.

What do you expect from the author of a class library that you are planning to reuse? Certainly you want good documentation, working code, well-specified interfaces, etc. But there are other things you want, too.

First of all, to make it worth your while to reuse this person's code, you want the author to guarantee to maintain it for you. After all, if *you* have to maintain it, you are going to have to invest a tremendous amount of time into it—time that might be better spent designing a smaller and better package for yourself.

Second, you are going to want the author to notify you in advance of any changes he plans to make to the interface and functionality of the code. But notification is not enough. The author must give you the option to refuse to use any new versions. After all, he might introduce a new version while you are in a severe schedule crunch, or he might make changes to the code that are simply incompatible with your system.

In either case, should you decide to reject his version, the author must guarantee to support your use of the old version for a time. Perhaps that time is as short as three months or as long as a year; that is something for the two of you to negotiate. But he can't just cut you loose and refuse to support you. If he won't agree to support your use of his older versions, then you may have to seriously consider whether you want to use his code and be subject to his capricious changes.

This issue is primarily political. It has to do with the clerical and support effort that must be provided if other people are going to reuse code. But those political and clerical issues have a profound effect on the packaging structure of software. In order to provide the guarantees that reusers need, authors must organize their software into reusable packages and then track those packages with release numbers.

The REP states that the granule of reuse (i.e., a package) can be no smaller than the granule of release. Anything that we reuse must also be released and tracked. It is not realistic for a developer to simply write a class and then claim it is reusable. Reusability comes only after there is a tracking system in place that offers the guarantees of notification, safety, and support that the potential reusers will need.

The REP gives us our first hint at how to partition our design into packages. Since reusability must be based on packages, reusable packages must contain reusable classes. So, at least some packages should comprise reusable sets of classes.

It may seem disquieting that a political force would affect the partitioning of our software, but software is not a mathematically pure entity that can be structured according to mathematically pure rules. Software is a human product that supports human endeavors. Software is created and used by humans. And if software is going to be reused, then it must be partitioned in a manner that humans find convenient for that purpose.

So what does this tell us about the internal structure of a package? One must consider the internal contents from the point of view of potential reusers. If a package contains software that should be reused, then it should not also contain software that is not designed for reuse. *Either all of the classes in a package are reusable or none of them are.*

Reusability is not the only criterion; we must also consider who the reuser is. Certainly, a container-class library is reusable, and so is a financial framework. But we would not want them to be part of the same package. There are many people who would like to reuse a container-class library who have no interest in a financial framework. Thus, we want all of the classes in a package to be reusable by the same audience. We do not want an audience to find that a package consists of some classes he needs and others that are wholly inappropriate for him.

The Common-Reuse Principle (CRP)

> *The classes in a package are reused together. If you reuse one of the classes in a package, you reuse them all.*

This principle helps us to decide which classes should be placed into a package. It states that classes that tend to be reused together belong in the same package.

Classes are seldom reused in isolation. Generally, reusable classes collaborate with other classes that are part of the reusable abstraction. The CRP states that these classes belong together in the same package. In such a package, we would expect to see classes that have lots of dependencies on each other.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus, they ought to be in the same package.

But the CRP tells us more than just what classes to put together into a package. It also tells us what classes *not* to put in the package. When one package uses another, a dependency is created between the packages. It may be that the using package only uses one class within the used package. However, that doesn't weaken the dependency at all. The using package still depends on the used package. Every time the used package is released, the using package must be revalidated and rereleased. This is true even if the used package is being released because of changes to a class that the using package doesn't care about.

Moreover, it is common for packages to have physical representations as shared libraries, DLLs, JARs. If the used package is released as a JAR, then the using code depends on the entire JAR. Any modification to that JAR—even if the modification is to a class that the using code does not care about—will still cause a new version of the JAR to be released. The new JAR will still have to be redistributed, and the using code will still have to be revalidated.

Thus, I want to make sure that when I depend on a package, I depend on every class in that package. To say this another way, I want to make sure that the classes that I put into a package are inseparable, that it is impossible to depend on some and not the others. Otherwise, I will be revalidating and redistributing more than is necessary, and I will waste significant effort.

Therefore, the CRP tells us more about what classes shouldn't be together than what classes should be together. The CRP says that classes which are not tightly bound to each other with class relationships should not be in the same package.

The Common-Closure Principle (CCP)

> *The classes in a package should be closed together against the same kinds of changes. A change that affects a package affects all the classes in that package and no other packages.*

This is the Single-Responsibility Principle restated for packages. Just as the SRP says that a class should not contain multiples reasons to change, this principle says that a package should not have multiple reasons to change.

In most applications, maintainability is more important that reusability. If the code in an application must change, you would rather that the changes occur all in one package, rather than being distributed through many packages. If changes are focused into a single package, then we need only release the one changed package. Other packages that don't depend on the changed package do not need to be revalidated or rereleased.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same package. This minimizes the workload related to releasing, revalidating, and redistributing the software.

This principle is closely associated with the Open–Closed Principle (OCP). For it is "closure" in the OCP sense of the word that this principle is dealing with. The OCP states that classes should be closed for modification but open for extension. But as we learned, 100% closure is not attainable. Closure must be strategic. We design our systems such that they are closed to the most common kinds of changes that we have experienced.

The CCP amplifies this by grouping together classes that are open to certain types of changes into the same packages. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of packages.

Summary of Package Cohesion

In the past, our view of cohesion was much simpler than the last three principles have implied. We used to think that cohesion was simply the attribute of a module to perform one, and only one, function. However, the three principles of package cohesion describe a richer variety of cohesion. In choosing the classes to group together into packages, we must consider the opposing forces involved in reusability and developability. Balancing these forces with the needs of the application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. Thus, the composition of the packages will likely jitter and evolve with time as the focus of the project changes from developability to reusability.

Stability: The Principles of Package Coupling

The next three principles deal with the relationships between packages. Here again, we will run into the tension between developability and logical design. The forces that impinge on the architecture of a package structure are technical, political, and volatile.



The Acyclic-Dependencies Principle (ADP)

> *Allow no cycles in the package-dependency graph.*

Have you ever worked all day, gotten some stuff working and then gone home, only to arrive the next morning at to find that your stuff no longer works? Why doesn't it work? Because somebody stayed later than you and changed something you depend on! I call this the "morning-after syndrome."

The morning-after syndrome occurs in development environments where many developers are modifying the same source files. In relatively small projects with just a few developers, it isn't too big a problem. But as the size of the project and the development team grows, the mornings after can get pretty nightmarish. It is not uncommon, in undisciplined teams, for weeks to go by without being able to build a stable version of the project. Instead, everyone keeps on changing and changing their code trying to make it work with the last changes that someone else made.

Over the last several decades, two solutions to this problem have evolved. Both solutions have come from the telecommunications industry. The first is "the weekly build," and the second is the ADP.

The Weekly Build

The weekly build is common in medium-sized projects. It works like this: All the developers ignore each other for the first four days of the week. They all work on private copies of the code and don't worry about integrating with each other. Then, on Friday, they integrate all their changes and build the system.

This has the wonderful advantage of allowing the developers to live in an isolated world for four days out of five. The disadvantage, of course, is the large integration penalty that is paid on Friday.

Unfortunately, as the project grows, it becomes less feasible to finish integrating on Friday. The integration burden grows until it starts to overflow into Saturday. A few such Saturdays are enough to convince the developers that integration should really begin on Thursday. And so the start of integration slowly creeps toward the middle of the week.

As the duty cycle of development vs. integration decreases, the efficiency of the team decreases, too. Eventually this becomes so frustrating that the developers, or the project managers, declare that the schedule should be changed to a biweekly build. This suffices for a time, but the integration time continues to grow with project size.

This eventually leads to a crisis. To maintain efficiency, the build schedule has to be continually lengthened. Yet lengthening the build schedule increases project risks. Integration and testing become harder and harder to do, and the team loses the benefit of rapid feedback.

Eliminating Dependency Cycles

The solution to this problem is to partition the development environment into releasable packages. The packages become units of work, which can be checked out by a developer or a team of developers. When developers get a package working, they release it for use by the other developers. They give it a release number and move it into a directory for other teams to use. They then continue to modify their package in their own private areas. Everyone else uses the released version.

As new releases of a package are made, other teams can decide whether or not to immediately adopt the new release. If they decide not to, they simply continue using the old release. Once they decide that they are ready, they begin to use the new release.

Thus, none of the teams is at the mercy of the others. Changes made to one package do not need to have an immediate effect on other teams. Each team can decide for itself when to adapt its packages to new releases of the packages they use. Moreover, integration happens in small increments. There is no single point in time when all developers must come together and integrate everything they are doing.

This is a very simple and rational process, and it is widely used. However, to make it work you must *manage* the dependency structure of the packages. *There can be no cycles.* If there are cycles in the dependency structure, then the morning-after syndrome cannot be avoided.

Consider the package diagram in Figure 20-1. Here we see a rather typical structure of packages assembled into an application. The function of this application is unimportant for the purpose of this example. What *is* important is the dependency structure of the packages. Notice that this structure is a *directed graph.* The packages are the *nodes*, and the dependency relationships are the *directed edges.*

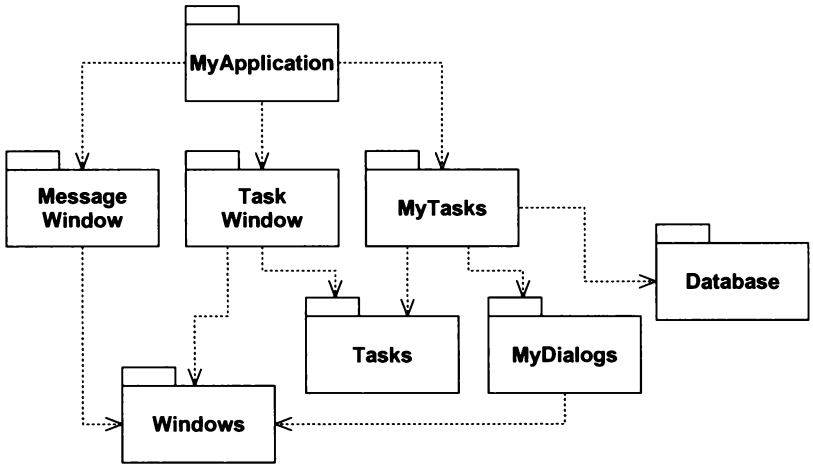


Figure 20-1 Package Structures are a Directed Acyclic Graph

Now notice one more thing. Regardless of the package at which you begin, it is impossible to follow the dependency relationships and wind up back at that package. This structures has no cycles. It is a *directed acyclic graph.* (DAG).

When the team responsible for `MyDialogs` makes a new release of their package it is easy to find out who is affected; you just follow the dependency arrows backwards. Thus, `MyTasks` and `MyApplication` are both going to be affected. The developers currently working on those packages will have to decide when they should integrate with the new release of `MyDialogs`.

Notice also that when `MyDialogs` is released, it has utterly no effect on many of the other packages in the system. They don't know about `MyDialogs`, and they don't care when it changes. This is nice. It means that the impact of releasing `MyDialogs` is relatively small.

When the developers working on the `MyDialogs` package would like to run a test of that package, all they need do is compile and link their version of `MyDialogs` with the version of the `Windows` package that they are currently using. None of the other packages in the system needs to be involved. This is nice; it means that the developers working on `MyDialogs` have relatively little work to do to set up a test, and that there are relatively few variables for them to consider.

When it is time to release the whole system, it is done from the bottom up. First the `Windows` package is compiled, tested, and released. Next are `MessageWindow` and `MyDialogs`. These are followed by `Task` and then `TaskWindow` and `Database`. `MyTasks` is next, and finally `MyApplication`. This process is very clear and easy to deal with. We know how to build the system because we understand the dependencies between its parts.

The Effect of a Cycle in the Package Dependency Graph

Let us say that the a new requirement forces us to change one of the classes in `MyDialogs` such that it makes use of a class in `MyApplication`. This creates a dependency cycle as shown in Figure 20-2.

This cycle creates some immediate problems. For example, the developers working on the `MyTasks` package know that in order to release, they must be compatible with `Task`, `MyDialogs`, `Database`, and `Windows`. However, with the cycle in place, they must now also be compatible with `MyApplication`, `TaskWindow`, and `MessageWindow`. That is, `MyTasks` now depends on *every other package in the system.* This makes `MyTasks` very difficult to release. `MyDialogs` suffers the same fate. In fact, the cycle forces `MyApplication`, `MyTasks`, and `MyDialogs` to always be released at the same time. They have, in effect, become one large package. And all the developers who are working in any of those packages will experience the morning-after syndrome once again.

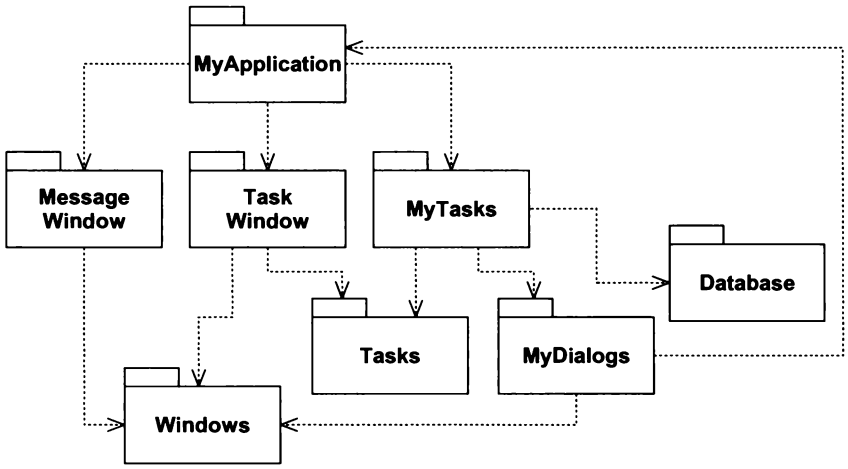


Figure 20-2 A Package Diagram with a Cycle

They will be stepping all over one another since they must all be using exactly the same release of each other's packages.

But this is just part of the trouble. Consider what happens when we want to test the `Mydialogs` package. We find that we must link in every other package in the system, including the `Database` package. This means that we have to do a *complete build* just to test `MyDialogs`. This is intolerable.

If you have ever wondered why you have to link in so many different libraries, and so much of everybody else's stuff, just to run a simple unit test of one of your classes, it is probably because there are cycles in the dependency graph. Such cycles make it very difficult to isolate modules. Unit testing and releasing become very difficult and error prone. And, in C++, compile times grow geometrically with the number of modules.

Moreover, when there are cycles in the dependency graph, it can be very difficult to work out the order in which to build the packages. Indeed, there may be no correct order. This can lead to some very nasty problems in languages like Java that read their declarations from compiled binary files.

Breaking the Cycle

It is always possible to break a cycle of packages and reinstate the dependency graph as a DAG. There are two primary mechanisms.

1. Apply the Dependency-Inversion Principle (DIP). In the case of Figure 20-3, we could create an abstract base class that has the interface that `MyDialogs` needs. We could then put that abstract base into `MyDialogs` and inherit it into `MyApplication`. This inverts the dependency between `MyDialogs` and `MyApplication`, thus breaking the cycle. (See Figure 20-3.)

   Notice, once again, that we named the interface after the client rather than the server. This is yet another application of the rule that interfaces belong to clients.
2. Create a new package on which both `MyDialogs` and `MyApplication` depend. Move the class(es) that they both depend on into that new package. (See Figure 20-4.)

The "Jitters"

The second solution implies that the package structure is volatile in the presence of changing requirements. Indeed, as the application grows, the package dependency structure jitters and grows. Thus, the dependency structure must always be monitored for cycles. When cycles occur, they must be broken somehow. Sometimes this will mean creating new packages, making the dependency structure grow.

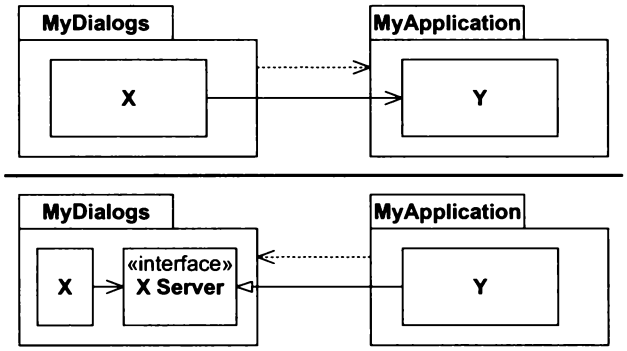


Figure 20-3 Breaking the cycle with dependency inversion

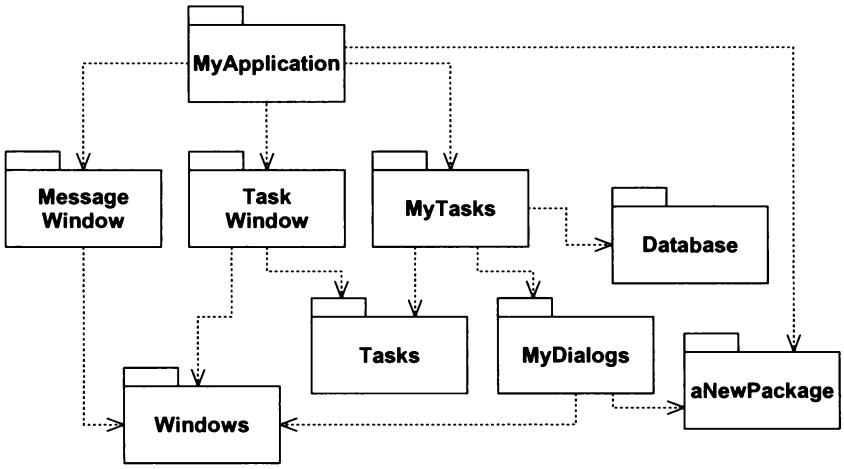


Figure 20-4 Breaking the cycle with a new package

Top-Down Design

The issues we have discussed so far lead to an inescapable conclusion. The package structure cannot be designed from the top down. This means that it is not one of the first things about the system that is designed. Indeed, it seems that it evolves as the system grows and changes.

You may find this to be counterintuitive. We have come to expect that large-grained decompositions, like packages, are also high-level *functional* decompositions. When we see a large-grained grouping like a package dependency structure, we feel that the packages ought to somehow represent the functions of the system. Yet this does not seem to be an attribute of package dependency diagrams.

In fact, package dependency diagrams have very little do to with describing the function of the application. Instead, they are a map to the *buildability* of the application. This is why they aren't designed at the start of the project. There is no software to build, and so there is no need for a build map. But as more and more classes accumulate in the early stages of implementation and design, there is a growing need to manage the dependencies so that the project can be developed without the morning-after syndrome. Moreover, we want to keep changes as localized as possible, so we start paying attention to the SRP and CCP and collocate classes that are likely to change together.

As the application continues to grow, we start becoming concerned about creating reusable elements. Thus, the CRP begins to dictate the composition of the packages. Finally, as cycles appear, the ADP is applied and the package dependency graph jitters and grows.

If we were to try to design the package dependency structure before we had designed any classes, we would likely fail rather badly. We would not know much about common closure, we would be unaware of any reusable elements, and we would almost certainly create packages that produce dependency cycles. Thus, the package dependency structure grows and evolves with the logical design of the system.

The Stable-Dependencies Principle (SDP)

Depend in the direction of stability.

Designs cannot be completely static. Some volatility is necessary if the design is to be maintained. We accomplish this by conforming to the Common-Closure Principle (CCP). Using this principle, we create packages that are sensitive to certain kinds of changes. These packages are *designed* to be volatile. We *expect* them to change.

Any package that we expect to be volatile should not be depended on by a package that is difficult to change! Otherwise the volatile package will also be difficult to change.

It is the perversity of software that a module that you have designed to be easy to change can be made hard to change by someone else simply hanging a dependency on it. Not a line of source code in your module need change, and yet your module will suddenly be hard to change. By conforming to the SDP, we ensure that modules that are intended to be easy to change are not depended on by modules that are harder to change than they are.

Stability

Stand a penny on its side. Is it stable in that position? You'd likely say that it was not. However, unless disturbed, it will remain in that position for a very long time. Thus, stability has nothing directly to do with frequency of change. The penny is not changing, but it is hard to think of it as stable.

Webster says that something is stable if it is "not easily moved."[1] Stability is related to the amount of work required to make a change. The penny is not stable because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of effort to turn it over.

How does this relate to software? There are many factors that make a software package hard to change: its size, complexity, clarity, etc. We are going to ignore all those factors and focus on something different. One sure way to make a software package difficult to change is to make lots of other software packages depend on it. A package with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent packages.

Figure 20-5 shows X, a stable package. This package has three packages depending on it; and therefore, it has three good reasons not to change. We say that it is *responsible* to those three packages. On the other hand, X depends on nothing, so it has no external influence to make it change. We say it is *independent*.

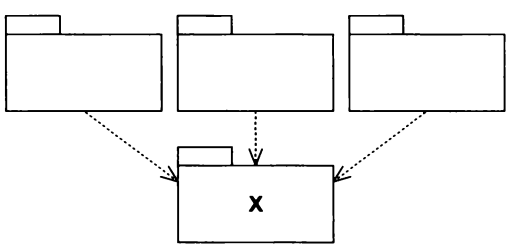


Figure 20-5 X: A Stable Package

1. *Webster's Third New International Dictionary.*

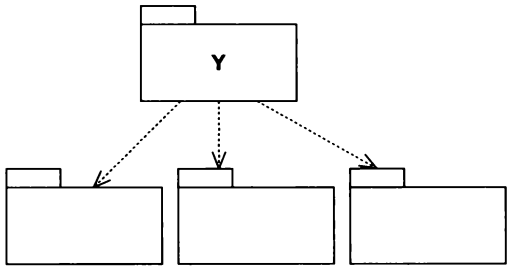


Figure 20-6 Y: An Instable Package

Figure 20-6, on the other hand, shows a very instable package. Y has no other packages depending on it; we say that it is irresponsible. Y also has three packages that it depends on, so changes may come from three external sources. We say that Y is dependent.

Stability Metrics

How can we measure the stability of a package? One way is to count the number of dependencies that enter and leave that package. These counts will allow us to calculate the *positional* stability of the package.

- (C_a) Afferent Couplings: The number of classes outside this package that depend on classes within this package.
- (C_e) Efferent Couplings: The number of classes inside this package that depend on classes outside this package.
- (Instability I)

$$I = \frac{C_e}{C_a + C_e}$$

This metric has the range [0,1]. $I = 0$ indicates a maximally stable package. $I = 1$ indicates a maximally instable package.

The C_a and C_e metrics are calculated by counting the number of *classes* outside the package in question that have dependencies on the classes inside the package in question. Consider the example in Figure 20-7.

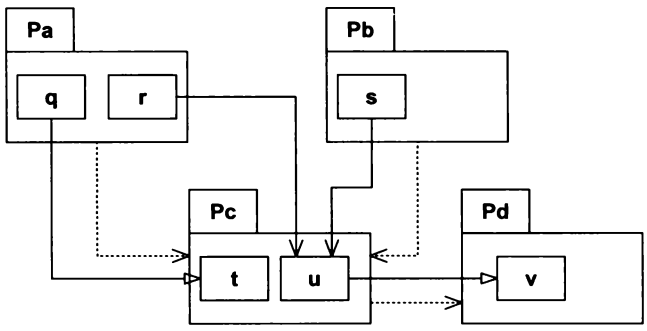


Figure 20-7 Tabulating C_a, C_e, and I.

The dashed arrows between the package represent package dependencies. The relationships between the classes of those packages show how those dependencies are actually implemented. There are inheritance and association relationships.

Now, let's say we want to calculate the stability of the package `Pc`. We find that there are three classes outside `Pc` that depend on classes in `Pc`. Thus $C_a = 3$. Moreover, there is one class outside `Pc` that classes in `Pc` depend on. Thus, $C_e = 1$, and $I = 1/4$.

In C++, these dependencies are typically represented by `#include` statements. Indeed, the *I* metric is easiest to calculate when you have organized your source code such that there is one class in each source file. In Java, the *I* metric can be calculated by counting import statements and qualified names.

When the *I* metric is 1, it means that no other package depends on this package ($C_a = 0$); and this package does depend on other packages ($C_e > 0$). This is as instable as a package can get; it is *irresponsible* and *dependent*. Its lack of dependents gives it no reason *not* to change, and the packages that it depends on may give it ample reason *to* change.

On the other hand, when the *I* metric is zero it means that the package is depended on by other packages ($C_a > 0$), but does not itself depend on any other packages ($C_e = 0$). It is *responsible* and *independent*. Such a package is as stable as it can get. Its dependents make it hard to change, and it has no dependencies that might force it to change.

The SDP says that the *I* metric of a package should be larger than the *I* metrics of the packages that it depends on (i.e., *I* metrics should decrease in the direction of dependency).

Not All Packages Should Be Stable

If all the packages in a system were maximally stable, the system would be unchangeable. This is not a desirable situation. Indeed, we want to design our package structure so that some packages are instable and some are stable. Figure 20-8 shows an ideal configuration for a system with three packages.

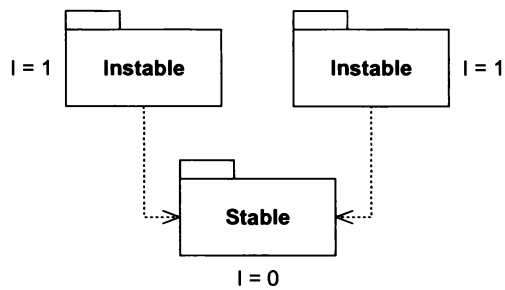


Figure 20-8 Ideal package configuration

The changeable packages are on top and depend on the stable package at the bottom. Putting the instable packages at the top of the diagram is a useful convention since any arrow that points *up* is violating the SDP.

Figure 20-9 shows how the SDP can be violated. `Flexible` is a package that we intend to be easy to change. We want `Flexible` to be instable. However, some developer, working in the package named `Stable`, hung a dependency on `Flexible`. This violates the SDP since the *I* metric for `Stable` is much lower than the *I* metric for `Flexible`. As a result, `Flexible` will no longer be easy to change. A change to `Flexible` will force us to deal with `Stable` and all its dependents.

To fix this, we somehow have to break the dependence of `Stable` on `Flexible`. Why does this dependency exist? Let's assume that there is a class `C` within `Flexible`, that another class `U` within `Stable` needs to use. (See Figure 20-10.)

We can fix this by employing the DIP. We create an interface class called `IU` and put it in a package named `UInterface`. We make sure that this interface declares all the methods that `U` needs to use. We then make `C` inherit from this interface. (See Figure 20-11.) This breaks the dependency of `Stable` on `Flexible` and forces both packages to be dependent on `UInterface`. `UInterface` is very stable ($I = 0$), and `Flexible` retains its necessary instability ($I = 1$). All the dependencies now flow in the direction of *decreasing I*.

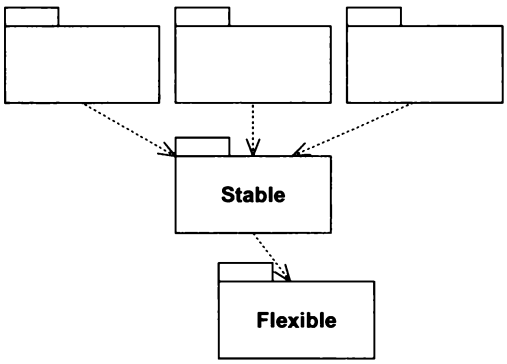


Figure 20-9 Violation of SDP

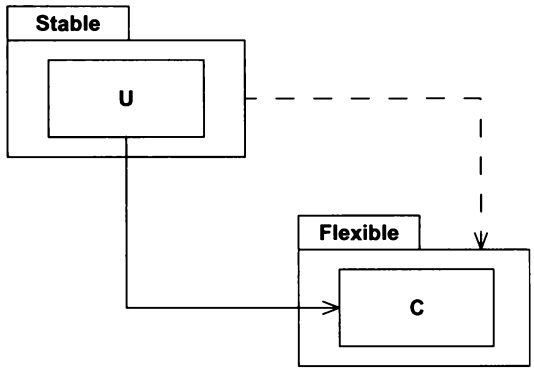


Figure 20-10 The cause of the bad dependency

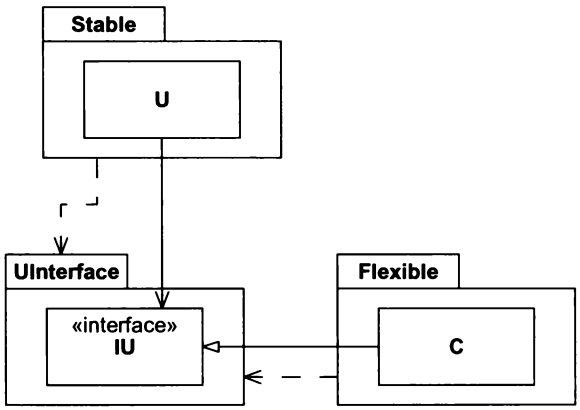


Figure 20-11 Fixing the stability violation using DIP

Where Do We Put the High-level Design?

Some software in the system should not change very often. This software represents the high-level architecture and design decisions. We don't want these architectural decisions to be volatile. Thus, the software that encapsulates the high-level design of the system should be placed into stable packages ($I = 0$). The instable packages ($I = 1$) should only contain the software that is likely to change.

However, if the high-level design is placed into stable packages, then the source code that represents that design will be difficult to change. This could make the design inflexible. How can a package that is maximally stable ($I = 0$) be flexible enough to withstand change? The answer is to be found in the OCP. This principle tells us that it is possible and desirable to create classes that are flexible enough to be extended without requiring modification. What kind of classes conforms to this principle? *Abstract* classes.

The Stable-Abstractions Principle (SAP)

> *A package should be as abstract as it is stable.*

This principle sets up a relationship between stability and abstractness. It says that a stable package should also be abstract so that its stability does not prevent it from being extended. On the other hand, it says that an instable package should be concrete since its instability allows the concrete code within it to be easily changed.

Thus, if a package is to be stable, it should also consist of abstract classes so that it can be extended. Stable packages that are extensible are flexible and do not overly constrain the design.

The SAP and the SDP combined amount to the DIP for packages. This is true because the SDP says that dependencies should run in the direction of stability, and the SAP says that stability implies abstraction. Thus, dependencies run in the direction of abstraction.

However, the DIP is a principle that deals with classes. And with classes there are no shades of grey. Either a class is abstract or it is not. The combination of the SDP and SAP deals with packages and allows that a package can be partially abstract and partially stable.

Measuring Abstraction

The *A* metric is a measure of the abstractness of a package. Its value is simply the ratio of abstract classes in a package to the total number of classes in the package.

N_c—The number of classes in the package.

N_a—The number of abstract classes in the package. Remember, an abstract class is a class with at least one pure interface, and it cannot be instantiated.

A—Abstractness.

$$A = \frac{N_a}{N_c}$$

The *A* metric ranges from 0 to 1. Zero implies that the package has no abstract classes at all. A value of 1 implies that the package contains nothing but abstract classes.

The Main Sequence

We are now in a position to define the relationship between stability (*I*) and abstractness (*A*). We can create a graph with *A* on the vertical axis and *I* on the horizontal axis. If we plot the two "good" kinds of packages on this graph, we will find the packages that are maximally stable and abstract at the upper left at (0,1). The packages that are maximally instable and concrete are at the lower right at (1,0). (See Figure 20-12.)

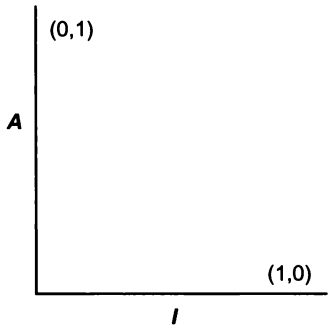


Figure 20-12 The *A*–*I* Graph

Not all packages can fall into one of these two positions. Packages have degrees of abstraction and stability. For example, it is very common for one abstract class to derive from another abstract class. The derivative is an abstraction that has a dependency. Thus, though it is maximally abstract, it will not be maximally stable. Its dependency will decrease its stability.

Since we cannot enforce that all packages sit at either (0,1) or (1,0), we must assume that there is a locus of points on the *A*/*I* graph that defines reasonable positions for packages. We can infer what that locus is by finding the areas where packages should *not* be (i.e., zones of *exclusion*). (See Figure 20-13.)

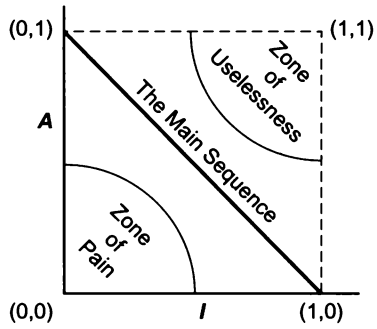


Figure 20-13 Zones of Exclusion

Consider a package in the area of (0,0). This is a highly stable and concrete package. Such a package is not desirable because it is rigid. It cannot be extended because it is not abstract. And it is very difficult to change because of its stability. Thus, we do not normally expect to see well-designed packages sitting near (0,0). The area around (0,0) is a zone of exclusion called the *Zone of Pain*.

It should be noted that there are cases when packages do indeed fall within the Zone of Pain. An example would be a database schema. Database schemas are notoriously volatile, extremely concrete, and highly depended on. This is one of the reasons that the interface between OO applications and databases is so difficult and that schema updates are generally painful.

Another example of a package that sits on (0,0) is a package that holds a concrete utility library. Although such a package has an *I* metric of 1, it may in fact be nonvolatile. Consider a "string" package for example. Even though all the classes within it are concrete, it is nonvolatile. Such packages are harmless in the (0,0) zone since they are not likely to be changed. Indeed, we can consider a third axis of the graph being that of volatility. If so, the graph in Figure 20-13 shows the plane at volatility = 1.

Consider a package near (1,1). This location is undesirable because it is maximally abstract and yet has no dependents. Such packages are useless. Thus, this is called the *Zone of Uselessness*.

It seems clear that we'd like our volatile packages to be as far from both zones of exclusion as possible. The locus of points that is maximally distant from each zone is the line that connects (1,0) and (0,1). This line is known as the *main sequence*.[2]

A package that sits on the main sequence is not "too abstract" for its stability, nor is it "too instable" for its abstractness. It is neither useless, nor particularly painful. It is depended on to the extent that it is abstract, and it depends on others to the extent that it is concrete.

Clearly, the most desirable positions for a package to hold are at one of the two endpoints of the main sequence. However, in my experience less than half the packages in a project can have such ideal characteristics. Those other packages have the best characteristics if they are on or close to the main sequence.

Distance from the Main Sequence

This leads us to our last metric. If it is desirable for packages to be on or close to the main sequence, we can create a metric which measures how far away a package is from this ideal.

D—Distance.

$$D = \frac{|A + I - 1|}{\sqrt{2}}.$$

This metric ranges from [0,~0.707].

2. The name "main sequence" was adopted because of my interest in astronomy and HR diagrams.

D' —Normalized Distance.

$$D' = |A + I - 1| .$$

This metric is much more convenient than D since it ranges from [0,1]. Zero indicates that the package is directly on the main sequence. One indicates that the package is as far away as possible from the main sequence.

Given this metric, a design can be analyzed for its overall conformance to the main sequence. The D metric for each package can be calculated. Any package that has a D value that is not near zero can be reexamined and restructured. In fact, this kind of analysis has been a great aid to the author in helping to define packages that are more maintainable and less sensitive to change.

Statistical analysis of a design is also possible. One can calculate the mean and variance of all the D metrics for the packages within a design. One would expect a conformant design to have a mean and variance close to zero. The variance can be used to establish "control limits," which can identify packages that are "exceptional" in comparison to all the others. (See Figure 20-14.)

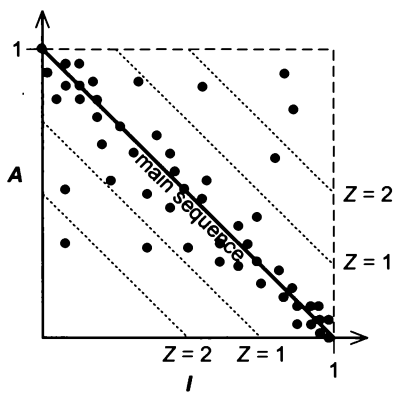


Figure 20-14 Scatter plot of Package D scores

In this scatter plot,[3] we see that the bulk of the packages lies along the main sequence, but some of them are more than one standard deviation ($Z = 1$) away from the mean. These aberrant packages are worth looking at. For some reason, they are either very abstract with few dependents or very concrete with many dependents.

Another way to use the metrics is to plot the D' metric of each package over time. Figure 20-15 shows a mock-up of such a plot. You can see that some strange dependencies have been creeping into the `Payroll` package over the last few releases. The plot shows a control threshold at $D' = 0.1$. The R2.1 point has exceeded this control limit, so it would be worth our while to find out why this package is so far from the main sequence.



Figure 20-15 Time plot of a single package's D' scores

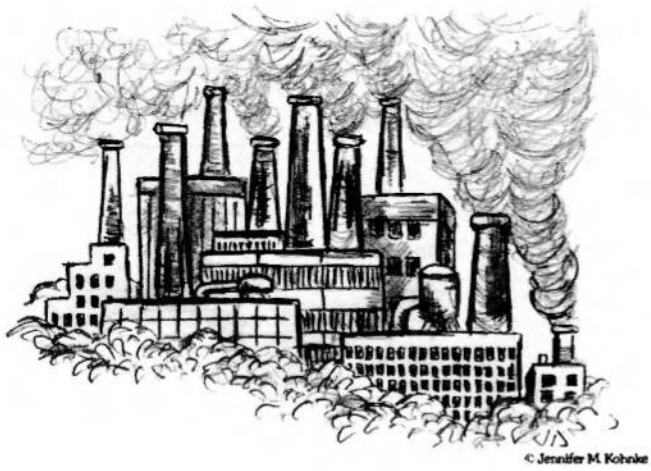
3. Not based on real data.

Conclusion

The *dependency-management metrics* described in this chapter measure the conformance of a design to a pattern of dependency and abstraction that I think is a "good" pattern. Experience has shown that certain dependencies are good and others are bad. This pattern reflects that experience. However, a metric is not a god; it is merely a measurement against an arbitrary standard. It is certainly possible that the standard chosen in this chapter is appropriate only for certain applications and is not appropriate for others. It may also be that there are far better metrics that can be used to measure the quality of a design.

21

FACTORY



The man who builds a factory builds a temple....

—Calvin Coolidge (1872–1933)

The Dependency-Inversion Principle (DIP)[1] tells us that we should prefer dependencies on abstract classes and avoid dependencies on concrete classes, especially when those classes are volatile. Therefore, the following snippet of code violates this principle:

```
Circle c = new Circle(origin, 1);
```

`Circle` is a concrete class. Therefore, those modules that create instances of `Circle` must violate DIP. Indeed, any line of code that uses the `new` keyword violates DIP.

There are times when violating the DIP is mostly harmless.[2] The more likely a concrete class is to change, the more likely depending on it will lead to trouble. But if the concrete class is not volatile, then depending on it is not worrisome.

For example, creating instances of `String` does not bother me. Depending on `String` is very safe because `String` is not likely to change any time soon.

1. "DIP: The Dependency-Inversion Principle" on page 127.
2. That's pretty good coverage.

On the other hand, when we are actively developing an application, there are many concrete classes that are very volatile. Depending on them is problematic. We'd rather depend on an abstract interface to shield us from the majority of the changes.

The FACTORY pattern allows us to create instances of concrete objects while depending only on abstract interfaces. Therefore, it can be of great assistance during active development when those concrete classes are highly volatile.

Figure 21-1 shows the problematic scenario. We have a class named `SomeApp` that depends on the interface `Shape`. `SomeApp` uses instances of `Shape` solely through the `Shape` interface. It does not use any of the specific methods of `Square` or `Circle`. Unfortunately, `SomeApp` also creates instances of `Square` and `Circle` and thus has to depend on the concrete classes.

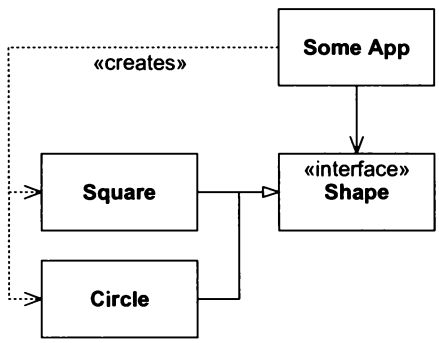


Figure 21-1 An app that violates the DIP to create concrete classes

We can fix this by applying the FACTORY pattern to `SomeApp` as in Figure 21-2. Here we see the `ShapeFactory` interface. This interface has two methods: `makeSquare` and `makeCircle`. The `makeSquare` method returns an instance of a `Square`, and the `makeCircle` method returns an instance of a `Circle`. However, the return type of both functions is `Shape`.

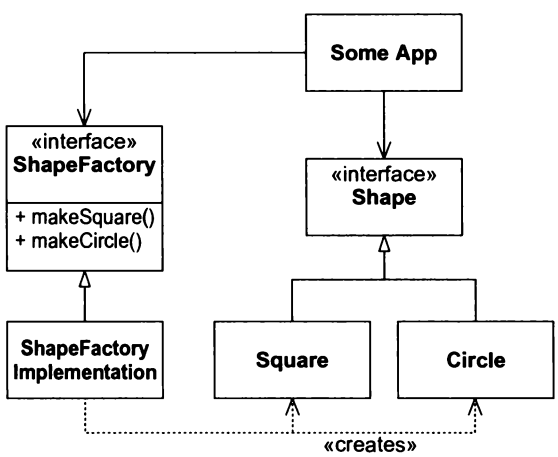


Figure 21-2 Shape Factory

Listing 21-1 shows what the `ShapeFactory` code looks like, and Listing 21-2 shows `ShapeFactory-Implementation`.

Listing 21-1

ShapeFactory.java

```
public interface ShapeFactory
{
  public Shape makeCircle();
  public Shape makeSquare();
}
```

Listing 21-2

ShapeFactoryImplementation.java

```
public class ShapeFactoryImplementation implements ShapeFactory
{
  public Shape makeCircle()
  {
    return new Circle();
  }

  public Shape makeSquare()
  {
    return new Square();
  }
}
```

Notice that this completely solves the problem of depending on concrete classes. The application code no longer depends on `Circle` or `Square`, and yet it still manages to create instances of them. It manipulates those instances through the `Shape` interface and never invokes methods that are specific to `Square` or `Circle`.

The problem of depending on a concrete class has been moved. Someone must create `ShapeFactoryImplementation`, but nobody else ever needs to create `Square` or `Circle`. `ShapeFactoryImplementation` will most likely be created by `main` or by an initialization function attached to `main`.

A Dependency Cycle

Astute readers will recognize that there is a problem with this form of the FACTORY pattern. The class `ShapeFactory` has a method for each of the derivatives of `Shape`. This results in a dependency cycle that makes it difficult to add new derivatives to `Shape`. Every time we add a new `Shape` derivative, we have to add a method to the `ShapeFactory` interface. In most cases, this means we'll have to recompile and redeploy all the users of `ShapeFactory`.[3]

We can get rid of this dependency cycle by sacrificing a little type safety. Instead of giving `ShapeFactory` one method for every `Shape` derivative, we can give it just one `make` function that takes a `String`. For example, look at Listing 21-3. This technique requires that `ShapeFactoryImplementation` use an `if/else` chain on the incoming argument to select which derivative of `Shape` to instantiate. This is shown in Listings 21-4 and 21-5.

3. Again, this isn't exactly necessary in Java. You might get away without recompiling and redeploying clients of a changed interface, but it's a risky business.

Listing 21-3

A snippet that creates a circle

```
public void testCreateCircle() throws Exception
{
  Shape s = factory.make("Circle");
  assert(s instanceof Circle);
}
```

Listing 21-4

ShapeFactory.java

```
public interface ShapeFactory
{
  public Shape make(String shapeName) throws Exception;
}
```

Listing 21-5

ShapeFactoryImplementation.java

```
public class ShapeFactoryImplementation implements ShapeFactory
{
  public Shape make(String shapeName) throws Exception
  {
    if (shapeName.equals("Circle"))
      return new Circle();
    else if (shapeName.equals("Square"))
      return new Square();
    else
      throw new Exception(
        "ShapeFactory cannot create " + shapeName);
  }
}
```

One might argue that this is dangerous because callers who misspell the name of a shape will get a run-time error instead of a compile time error. This is true. However, if you are writing the appropriate number of unit tests and applying test-driven development, then you'll catch these run-time errors long before they become problems.

Substitutable Factories

One of the great benefits of using factories is the ability to substitute one implementation of a factory for another. In this way, you can substitute families of objects within an application.

For example, imagine an application that had to adapt to many different database implementations. In our example, let's assume that the users can either use flat files or they can purchase an Oracle™ adapter. We might use the PROXY[4] pattern to isolate the application from the database implementation. We might also use factories to instantiate the proxies. Figure 21-3 shows the structure.

Notice that there are two implementations of `EmployeeFactory`. One creates proxies that work with flat files, and the other creates proxies that work with Oracle™. Notice also that the application does not know or care which is being used.

4. We'll study PROXY later on page 327. Right now, all you need to know is that a PROXY is a class that knows how to read particular objects out of particular kinds of databases.

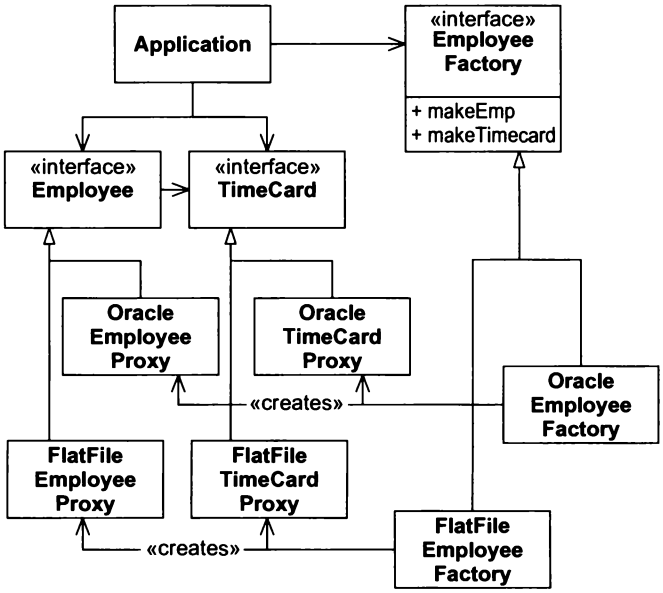


Figure 21-3 Substitutable Factory

Using Factories for Test Fixtures

When writing unit tests, we often want to test the behavior of a module in isolation from the modules it uses. For example, we might have a `Payroll` application that uses a database. (See Figure 21-4.) We may wish to test the function of the `Payroll` module without using the database at all.

We can accomplish this by using an abstract interface for the database. One implementation of this abstract interface uses the real database. Another implementation is test-code written to simulate the behavior of the database and to check that the database calls are being made correctly. Figure 21-5 shows the structure. The `PayrollTest` module tests the `PayrollModule` by making calls to it. It also implements the `Database` interface so that it can trap the calls that `Payroll` makes to the database. This allows `PayrollTest` to ensure that `Payroll` is behaving properly. It also allows `PayrollTest` to simulate many kinds of database failures and problems that are otherwise difficult to create. This is a technique that is sometimes known as *spoofing*.

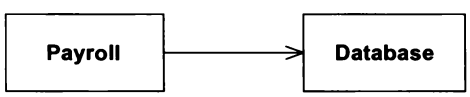


Figure 21-4 Payroll uses Database

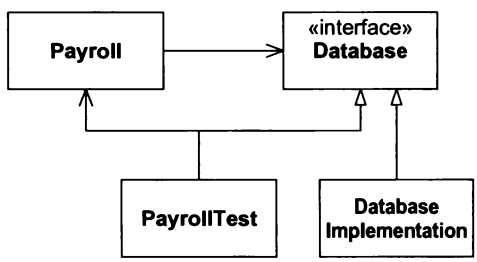


Figure 21-5 `PayrollTest` Spoofs Database

However, how does `Payroll` get the instance of `PayrollTest` it uses as the `Database`. Certainly `Payroll` isn't going to do the creation of `PayrollTest`. Just as clearly, `Payroll` must somehow get a reference to the `Database` implementation it's going to use.

In some cases, it is perfectly natural for `PayrollTest` to pass the `Database` reference to `Payroll`. In other cases, it may be that `PayrollTest` must set a global variable to refer to the `Database`. In still others, `Payroll` may be fully expecting to create the `Database` instance. In that last case, we can use a `Factory` to fool `Payroll` into creating the test version of the `Database` by passing an alternate factory to `Payroll`.

Figure 21-6 shows a possible structure. The `Payroll` module acquires the factory through a global variable (or a static variable in a global class) named `GdatabaseFactory`. The `PayrollTest` module implements `DatabaseFactory` and sets a reference to itself into that `GdatabaseFactory`. When `Payroll` uses the factory to create a `Database`, the `PayrollTest` module traps the call and passes back a reference to itself. Thus, `Payroll` is convinced that it has created the `PayrollDatabase`, and yet the `PayrollTest` module can fully spoof the `Payroll` module and trap all database calls.

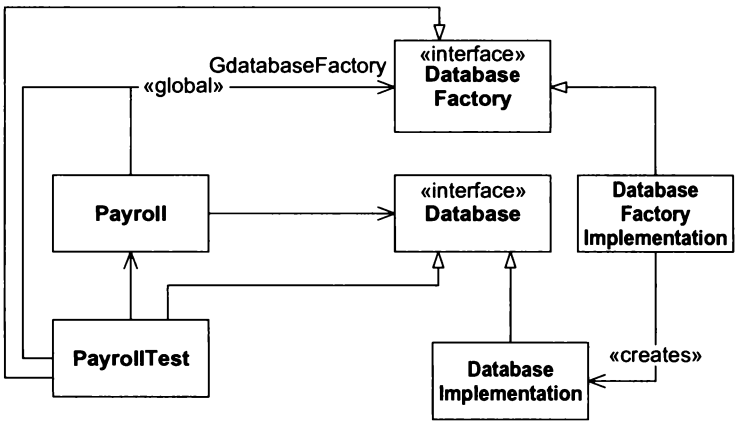


Figure 21-6 Spoofing the Factory

How Important Is It to Use Factories?

A strict interpretation of the DIP would insist on using factories for every volatile class in the system. What's more, the power of the FACTORY pattern is seductive. These two factors can sometimes seduce developers into using factories by default. This is an extreme that I don't recommend.

I don't start out using factories. I only put them into the system when the need for them becomes great enough. For example, if it becomes necessary to use the PROXY pattern, then it will probably become necessary to use a factory to create the persistent objects. Or if, through unit testing, I come across situations where I must spoof the creator of an object, then I will likely use a factory. But I don't start out assuming that factories will be necessary.

Factories are a complexity that can often be avoided, especially in the early phases of an evolving design. When they are employed by default, they dramatically increase the difficulty of extending the design. In order to create a new class, one may have to create as many as four new classes. The four are the two interface classes that represent the new class and its factory and the two concrete classes that implement those interfaces.

Conclusion

Factories are powerful tools. They can be of great benefit in conforming to the DIP. They allow high-level policy modules to create instances of classes without depending on the concrete implementations of those classes. They also make it possible to swap in completely different families of implementations for a group of classes. However, factories are a complexity that can often be avoided. Using them by default is seldom the best course of action.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

22

The Payroll Case Study (Part 2)



"Rule of thumb: if you think something is clever and sophisticated, beware—it is probably self-indulgence."

—Donald A. Norman, 1990
(*The Design of Everyday Things*, Donald A. Norman, Doubleday, 1990)

We have done a great deal of analysis, design, and implementation for the Payroll problem. However, we still have many decisions to make. For one thing, the number of programmers who have been working on the problem is—one (me). The current structure of the development environment is consistent with this. All the program files are located in a single directory. There is no higher order structure at all. There are no packages, no subsystems, no releasable units other than the entire application. This will not do going forward.

We must assume that as this program grows, the number of people working on it will grow too. In order to make it convenient for multiple developers, we are going to have to partition the source code into packages that can be conveniently checked out, modified, and tested.

The payroll application currently consists of 3280 lines of code divided into about 50 different classes and 100 different source files. Although this is not a huge number, it does represent an organizational burden. How should we manage these source files?

Along similar lines, how should we divide the work of implementation so that the development can proceed smoothly without the programmers getting in each other's way. We would like to divide the classes into groups that are convenient for individuals or teams to check out and support.